DISEÑO WEB EN ENTORNO CLIENTE · UT_04.1

Eventos en JavaScript (I)

Procesamiento y activación de eventos

```
boton.addEventListener('click', () => {
  console.log('¡Evento recibido!');
});
```

Contenido de la unidad

1

¿Qué es un evento?

Programación dirigida por eventos y bucle de eventos.

2

Responder a eventos

Atributos HTML, propiedades DOM y addEventListener().

3

El objeto evento

Propiedades, target frente a currentTarget.

4

Ratón y teclado

Coordenadas, Pointer Events, key frente a code.

Programación basada en eventos

El código no se ejecuta de arriba abajo y termina: queda a la espera y reacciona cuando ocurre algo.



¿Quién genera eventos?

La persona usuaria

Pulsa, escribe, arrastra, toca la pantalla.

`click`
`keydown`
`input`

El documento

Se carga, cambia o termina de pintarse.

`DOMContentLoaded`
`load`
`submit`

La ventana

Cambia de tamaño, se desplaza o pierde la conexión.

`resize`
`scroll`
`offline`

El propio código

Puede crear y lanzar eventos propios (UT_04.2).

`dispatchEvent()`
`CustomEvent`

Pila de llamadas y bucle de eventos

COLA DE EVENTOS

Los eventos esperan su turno, en orden de llegada.

`click · keydown · scroll`

→

BUCLE DE EVENTOS

Si la pila está vacía, toma el siguiente evento de la cola.

→

PILA DE LLAMADAS

Ejecuta el escuchador hasta que termina.

`manejarClick()`

JavaScript tiene un solo hilo: mientras un escuchador se ejecuta, nada más puede hacerlo.

Un escuchador muy lento bloquea la página: los eventos se acumulan en la cola.

Las microtareas (promesas) se ejecutan antes de atender el siguiente evento.

Responder a un evento en tres pasos

PASO 1

Elegir el elemento

PASO 2

Escribir la función

PASO 3

Asociarla al evento

```
// 1. Elemento
const boton = document.querySelector('#boton');
// 2. Función
const saludar = () => console.log('¡Hola!');
// 3. Asociación
boton.addEventListener('click', saludar);
```

Tres formas de asociar un controlador

1 · Atributo HTML

```
<button onclick="saludar()">
```

Desaconsejada

Mezcla HTML y JavaScript.

2 · Propiedad DOM

```
boton.onclick = saludar;
```

Limitada

Solo un controlador por evento.

3 · addEventListener()

```
boton.addEventListener(
  'click', saludar);
```

Recomendada

Varios escuchadores y opciones.

Atributos HTML: mejor evitarlos

```
<input type="number" id="edad"
  onblur="comprobarEdadNoNegativa()">
```

Mezcla estructura y comportamiento

Una política CSP estricta los bloquea

No ven las funciones de un módulo ES6

Propiedades del DOM: onclick, onblur…

```
const campoEdad = document.querySelector('#edad');

function comprobarEdadNoNegativa() {
  if (campoEdad.value < 0) alert('Edad no válida');
}

campoEdad.onblur = comprobarEdadNoNegativa;
```

JavaScript queda separado del HTML.

Sin paréntesis: se asigna la función, no se ejecuta.

Limitación: solo un controlador por evento

```
boton.onclick = () => console.log('Primero');
boton.onclick = () => console.log('Segundo');

// Al hacer clic solo se muestra: Segundo
```

onclick = A → **onclick = B** → **A se pierde**

addEventListener(): el método estándar

```
elemento.addEventListener(tipo, escuchador, opciones);
```

tipo

Nombre del evento, sin «on»: 'click', 'keydown'…

escuchador

Función que recibe el objeto evento.

opciones

Opcional: objeto con capture, once, passive, signal.

Opciones de addEventListener()

capture

Escucha en la fase de captura, de fuera hacia dentro.

once

El escuchador se ejecuta una sola vez y se elimina.

passive

Promete no llamar a preventDefault(): mejora el desplazamiento.

signal

Un AbortController permite quitar el escuchador.

```
boton.addEventListener('click', () => {
  console.log('Solo la primera vez');
}, { once: true });
```

Varios escuchadores y cómo quitarlos

```
const registrar = () => console.log('Registrado');
const avisar = () => console.log('Aviso');

boton.addEventListener('click', registrar);
boton.addEventListener('click', avisar);
// Ambos se ejecutan, en el orden en que se añadieron

boton.removeEventListener('click', avisar);
```

removeEventListener() necesita la misma función, el mismo tipo y la misma captura.

Una función anónima no se puede quitar así: usa un AbortController con signal.

Pasar argumentos con funciones flecha

El escuchador no admite parámetros propios; para pasarlos se envuelve la llamada en una función flecha.

Incorrecto: se ejecuta al instante

```
boton.addEventListener(
  'click', obtenerEdad(campo));
```

Correcto: se ejecuta al hacer clic

```
boton.addEventListener('click',
  () => obtenerEdad(campo));
```

```
const obtenerEdad = (campo) => Number(campo.value);
boton.addEventListener('click', () => console.log(obtenerEdad(campoEdad)));
```

¿Cuándo está lista la página?

`DOMContentLoaded`

El HTML está analizado y el DOM construido. No espera imágenes ni hojas de estilo. Se escucha en **document**.

`load`

Todo se ha cargado, incluidos recursos externos. Se escucha en **window**.

```
document.addEventListener('DOMContentLoaded', () => {
  console.log('DOM listo');
});
window.addEventListener('load', () => console.log('Todo cargado'));
```

Módulos ES6 y eventos

index.html

```
<script type="module"
  src="principal.js"></script>
```

Se ejecutan en diferido: el DOM ya está listo.

saludos.js

```
export const saludar = () => {
  console.log('¡Hola!');
};
```

Ámbito propio: sus funciones no son globales.

principal.js

```
import { saludar } from './saludos.js';
boton.addEventListener('click', saludar);
```

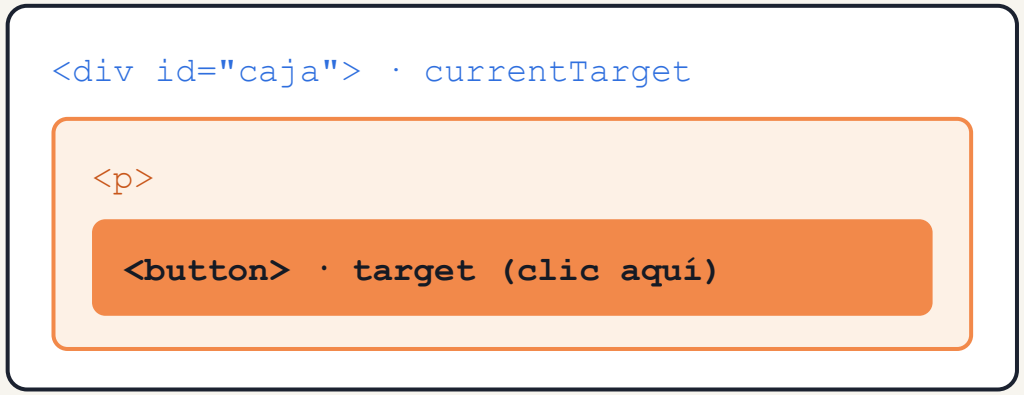
Por eso no valen los atributos onclick="…".

El objeto evento

```
boton.addEventListener('click', (evento) => {
  console.log(evento.type, evento.target);
});
```

- type · nombre del evento
- target · elemento que lo originó
- currentTarget · elemento con el escuchador
- timeStamp · instante, en milisegundos
- isTrusted · generado por el usuario
- cancelable · admite preventDefault()
- eventPhase · captura, destino o burbuja
- composedPath() · ruta de elementos

target frente a currentTarget



target: el elemento más interno donde ocurrió el evento.

currentTarget: el elemento al que está unido el escuchador. Coincide con **this** en una función clásica, no en una flecha.

```
caja.addEventListener('click', (evento) => {
  console.log(evento.target.tagName, evento.currentTarget.id); // BUTTON caja
});
```

Tipos de eventos

Ratón

`click` · `mousemove`

ESTA UNIDAD

Teclado

`keydown` · `keyup`

ESTA UNIDAD

Formulario

`submit` · `input` · `change`

UT_04.2

Táctil y puntero

`pointerdown` · `touchstart`

Ventana y documento

`load` · `resize` · `scroll`

Foco y portapapeles

`focus` · `blur` · `copy`

Eventos de ratón

- `click` · pulsar y soltar
- `dblclick` · doble clic
- `contextmenu` · botón derecho
- `auxclick` · botón central u otros
- `wheel` · rueda del ratón
- `mousedown` · al pulsar
- `mouseup` · al soltar
- `mousemove` · al moverse
- `mouseenter` / `mouseleave`
- `mouseover` / `mouseout`

```
area.addEventListener('contextmenu', (evento) => evento.preventDefault());
```

mouseenter / mouseleave frente a mouseover / mouseout

`mouseenter` · `mouseleave`

No burbujean

Solo se disparan al entrar o salir del propio elemento, no al pasar por sus hijos.

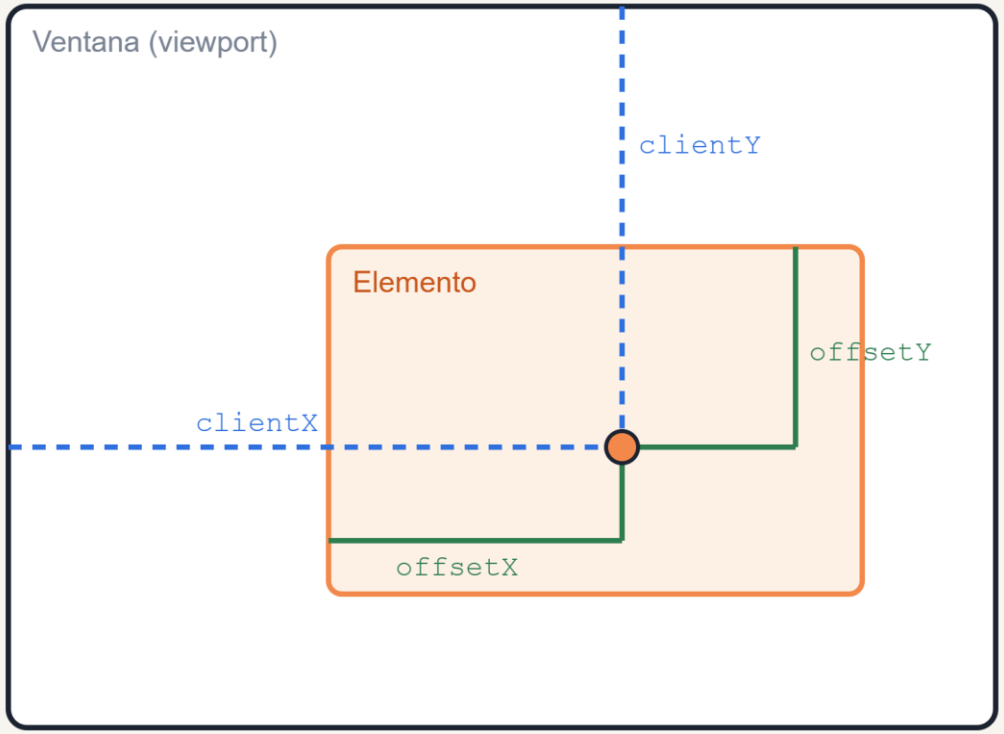
`mouseover` · `mouseout`

Sí burbujean

Se repiten cada vez que el puntero cruza un descendiente. Útiles para delegación de eventos.

```
tarjeta.addEventListener('mouseenter', () => tarjeta.classList.add('resaltada'));
tarjeta.addEventListener('mouseleave', () => tarjeta.classList.remove('resaltada'));
```

Coordenadas del puntero



`clientX / clientY`
Respecto a la ventana visible.

`pageX / pageY`
Respecto al documento completo, con desplazamiento.

`offsetX / offsetY`
Respecto al elemento sobre el que está el puntero.

Ejemplo: dibujar donde se hace clic

```
const lienzo = document.querySelector('#lienzo');
const contexto = lienzo.getContext('2d');

lienzo.addEventListener('click', (evento) => {
  contexto.beginPath();
  contexto.arc(evento.offsetX, evento.offsetY, 10, 0, Math.PI * 2);
  contexto.fill();
});
```

offsetX y offsetY coinciden con el sistema de coordenadas del lienzo.

Reto: usa mousemove para pintar mientras se mantiene pulsado el botón.

Pointer Events: un solo modelo para todo

Ratón, dedo y lápiz generan los mismos eventos. Es la opción moderna para interacciones táctiles.

`pointerdown`	`pointermove`	`pointerup`	**pointerType** **mouse · touch · pen**

```
pizarra.addEventListener('pointerdown', (evento) => {
  pizarra.setPointerCapture(evento.pointerId);
  console.log(evento.pointerType);
});
```

Con setPointerCapture() el elemento sigue recibiendo los eventos aunque el puntero salga de él. En CSS, touch-action: none evita que el navegador desplace la página.

Eventos de teclado

`keydown`

Al pulsar la tecla; se repite si se mantiene.

`keyup`

Al soltar la tecla.

~~`keypress`~~

Obsoleto: no usar.

`evento.key`

El carácter producido: depende de mayúsculas e idioma del teclado.

`'a' 'A' 'Enter'`

`evento.code`

La tecla física: no cambia con la distribución del teclado.

`'KeyA' 'Enter'`

Otras propiedades útiles: repeat, shiftKey, ctrlKey, altKey.

Ejemplo: mover un elemento con las flechas

```
let posicionX = 0;
let posicionY = 0;
document.addEventListener('keydown', (evento) => {
  switch (evento.key) {
    case 'ArrowLeft': posicionX -= 10; break;
    case 'ArrowRight': posicionX += 10; break;
    case 'ArrowUp': posicionY -= 10; break;
    case 'ArrowDown': posicionY += 10; break;
    default: return;
  }
  evento.preventDefault();
  personaje.style.transform = `translate(${posicionX}px, ${posicionY}px)`;
});
```

> preventDefault() evita que las flechas desplacen la página mientras se juega.

Resumen

Eventos

Un emisor genera un evento y un escuchador reacciona; el bucle los atiende de uno en uno.

addEventListener

La forma estándar: varios escuchadores y opciones como once o signal.

Sin paréntesis

Se pasa la función, no su resultado. Con argumentos, usar una función flecha.

Objeto evento

target es el origen; currentTarget, quien escucha.

Ratón y puntero

client, page y offset; Pointer Events para ratón, dedo y lápiz.

Teclado

keydown y keyup; key para el carácter, code para la tecla física.

CONTINÚA EN LA UT_04.2

Eventos en JavaScript (II)

- **Eventos de formulario y de foco**
- **Flujo: captura, destino y burbuja**
- **Delegación de eventos**
- **Eventos personalizados con CustomEvent**